

ASSIGNMENT No 2

```
import java.util.*;

public class TicTacToeHard {

    static char[][] board = new char[3][3];
    static char player = 'X';
    static char computer = 'O';

    public static void main(String[] args) {
        initializeBoard();
        playGame();
    }

    static void initializeBoard() {
        for (int i = 0; i < 3; i++)
            Arrays.fill(board[i], ' ');
    }

    static void printBoard() {
        System.out.println("\n-----");
        for (int i = 0; i < 3; i++) {
            System.out.print("| ");
            for (int j = 0; j < 3; j++) {
                System.out.print(board[i][j] + " | ");
            }
            System.out.println("\n-----");
        }
    }

    static void playGame() {
        Scanner sc = new Scanner(System.in);

        System.out.println("Tic Tac Toe - HARD LEVEL (Unbeatable AI)\n");

        // Let user choose symbol
        System.out.print("Choose your symbol (X/O): ");
        char choice = 'X';
        while (true) {
            String s = sc.next();
            if (s != null && s.length() > 0) {
                char c = Character.toUpperCase(s.charAt(0));
                if (c == 'X' || c == 'O') {
                    choice = c;
                    break;
                }
            }
        }
    }
}
```

```

    System.out.print("Invalid choice. Enter X or O: ");
}

player = choice;
computer = (player == 'X') ? 'O' : 'X';
System.out.println("You are " + player + ". Computer is " + computer + ".\n");

// If computer is X, it goes first
if (computer == 'X') {
    computerMove();
}

while (true) {

    printBoard();

    int row = -1, col = -1;

    // Safe input
    while (true) {
        try {
            System.out.print("Enter row (0-2): ");
            row = sc.nextInt();
            System.out.print("Enter col (0-2): ");
            col = sc.nextInt();

            if (row >= 0 && row < 3 && col >= 0 && col < 3 && board[row][col] == ' ')
                break;
            else
                System.out.println("Invalid move! Try again.");
        } catch (Exception e) {
            System.out.println("Invalid input! Enter numbers only.");
            sc.nextLine();
        }
    }

    board[row][col] = player;

    if (checkWin(player)) {
        printBoard();
        System.out.println("Player Wins!");
        break;
    }

    if (isBoardFull()) {
        printBoard();
        System.out.println("Game Draw!");
        break;
    }
}

```

```

    }

    computerMove();

    if (checkWin(computer)) {
        printBoard();
        System.out.println("Computer Wins!");
        break;
    }

    if (isBoardFull()) {
        printBoard();
        System.out.println("Game Draw!");
        break;
    }
}

sc.close();
}

// HARD AI USING MINIMAX
static void computerMove() {
    int bestScore = Integer.MIN_VALUE;
    int bestRow = -1, bestCol = -1;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = computer;
                int score = minimax(false);
                board[i][j] = ' ';
                if (score > bestScore) {
                    bestScore = score;
                    bestRow = i;
                    bestCol = j;
                }
            }
        }
    }

    board[bestRow][bestCol] = computer;
    System.out.println("Computer played at: " + bestRow + " " + bestCol);
}

static int minimax(boolean isMaximizing) {

    if (checkWin(computer)) return 1;
    if (checkWin(player)) return -1;

```

```

if (isBoardFull()) return 0;

if (isMaximizing) {
    int bestScore = Integer.MIN_VALUE;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = computer;
                int score = minimax(false);
                board[i][j] = ' ';
                bestScore = Math.max(score, bestScore);
            }
        }
    }
    return bestScore;
} else {
    int bestScore = Integer.MAX_VALUE;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = player;
                int score = minimax(true);
                board[i][j] = ' ';
                bestScore = Math.min(score, bestScore);
            }
        }
    }
    return bestScore;
}
}

static boolean checkWin(char p) {

    for (int i = 0; i < 3; i++)
        if (board[i][0] == p && board[i][1] == p && board[i][2] == p)
            return true;

    for (int i = 0; i < 3; i++)
        if (board[0][i] == p && board[1][i] == p && board[2][i] == p)
            return true;

    if (board[0][0] == p && board[1][1] == p && board[2][2] == p)
        return true;

    if (board[0][2] == p && board[1][1] == p && board[2][0] == p)
        return true;

    return false;
}

```

```

    }

    static boolean isBoardFull() {
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                if (board[i][j] == ' ')
                    return false;
        return true;
    }
}

```

OUTPUT:-

Tic Tac Toe - HARD LEVEL (Unbeatable AI)

Choose your symbol (X/O): O

You are 'O'. Computer is 'X'.

Computer played at: 0 0

```

-----
| X |  |  |
-----
|  |  |  |
-----
|  |  |  |
-----

```

Enter row (0-2): 1 0

Enter col (0-2): Computer played at: 0 1

```

-----
| X | X |  |
-----
| O |  |  |
-----
|  |  |  |
-----

```

Enter row (0-2): 0 3

Enter col (0-2): Invalid move! Try again.

Enter row (0-2): 0 2

Enter col (0-2): Computer played at: 1 1

| X | X | O |

| O | X | |

| | | |

Enter row (0-2): 2 2

Enter col (0-2): Computer played at: 2 1

| X | X | O |

| O | X | |

| | X | O |

Computer Wins!